

大前端稳定性研发范式重塑

柯南 AI Agent 工程实践

个人简介



王泽锋

快手 / 主站技术

- ◆ 23 年加入快手，参与 Android、鸿蒙稳定性全链路建设（监控、排障、止损）
- ◆ 目前聚焦探索 AI + 稳定性

目录

01

AI 对稳定性保障的冲击

为什么 · AI Native 最后一公里

02

柯南 AI 定位与产品形态

是什么 · 嵌入工作流的稳定性数字同事

03

柯南 AI 的工程关键

怎么做 · 五层架构 + 反馈迭代

04

总结与展望

看未来 · Agent First, Human in Control

01

AI 对稳定性保障的冲击

AI Native 最后一公里

AI Coding 提速 vs 稳定性保障滞后

↑ 上游 研发吞吐显著提升

Cursor / Claude Code / Codex / ...

需求 → 代码 转化成本压到极低

↓ 下游 稳定性处置链路几乎零加速

线上告警 / 风险识别

根因定位 / 止损决策

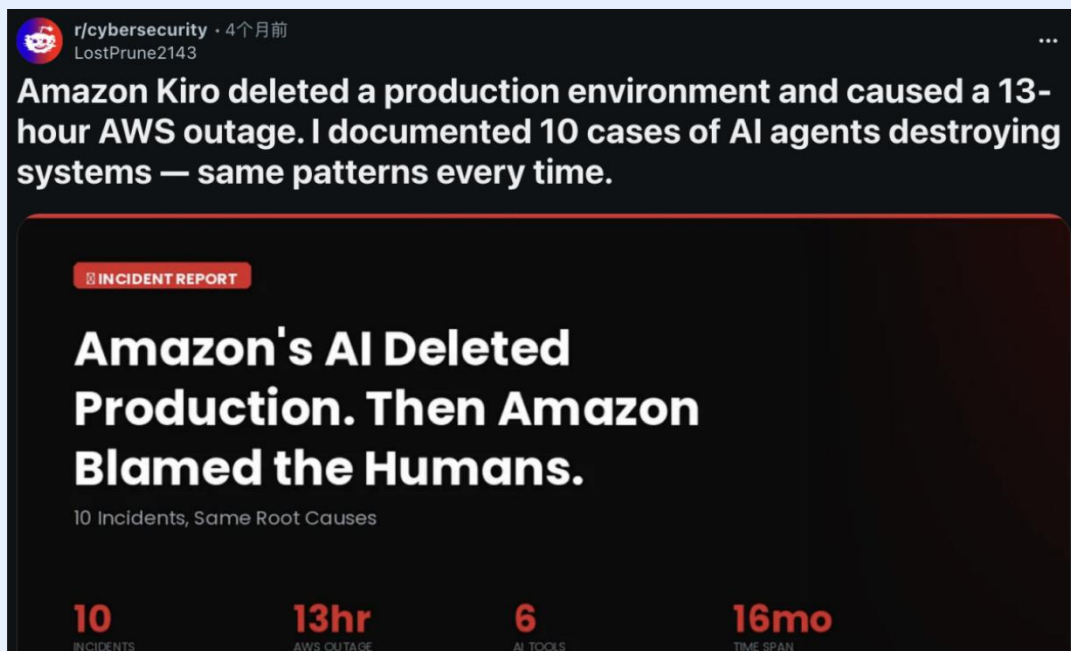
修复回归 / 经验沉淀

→ 线上 case 数量 / 严重度 / 时延
全面劣化风险

AI 在制造新问题 | 「局部最优陷阱」

AI 缺乏全局上下文 → 沿「最小阻力路径」让现象消失 ≠ 真正解决问题

外部 | Amazon 多起严重事故疑似 AI 相关



内部 | AI 生 ProGuard 配置致 DEX 膨胀 6MB

灰度 APK 突增 6MB · R8 全局优化关闭

SDK 自己工程
consumerProguardFiles
(SDK 跑得通)



主站 App 构建
-dontoptimize 进主工程
→ R8 全局关闭

AI 视角

只看到 SDK 自己跑得通的局部 · 对跨工程边界传递无概念

■ 前置拦截消灭不了所有问题 | 状态空间爆炸 + 不完备性

SDD / Coding Harness 等前置手段：能解决一部分 · 但消灭不了所有问题

OEM ROM 100+	设备型号 1000+	AB / 灰度 / 活动 10000+	用户行为路径 ∞
-----------------	---------------	------------------------	--------------------

组合空间轻松突破 10^9 量级 · 没有任何测试矩阵能完全覆盖

软件稳定性 = 不完备性证明 · 线上真实现场永远是稳定性的核心战场

AI Native 的最后一公里

研发吞吐被 AI 拉快

+

处置侧没跟上

+

AI 局部最优陷阱

+

前置消灭不了



线上稳定性处置 = AI Native 落地的最后一公里



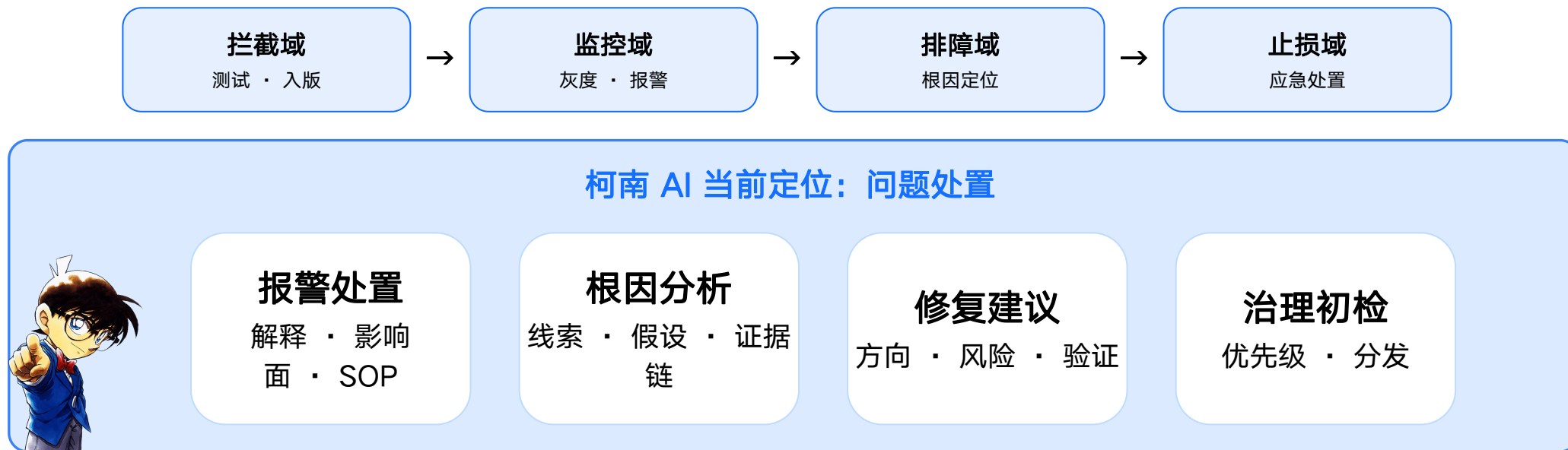
用 AI 打败 AI

02

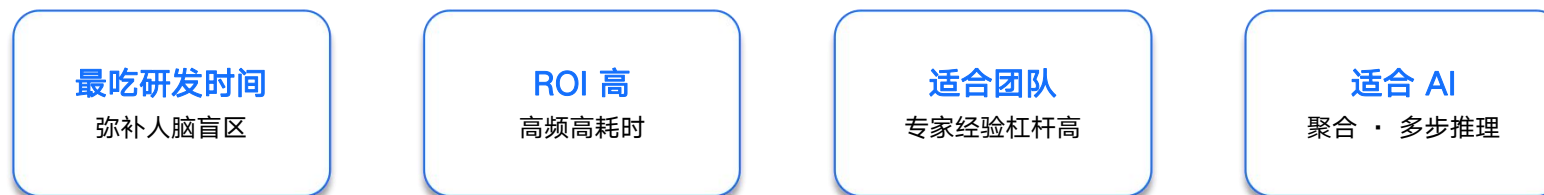
柯南 AI 定位与产品形态

嵌入工作流的稳定性数字同事

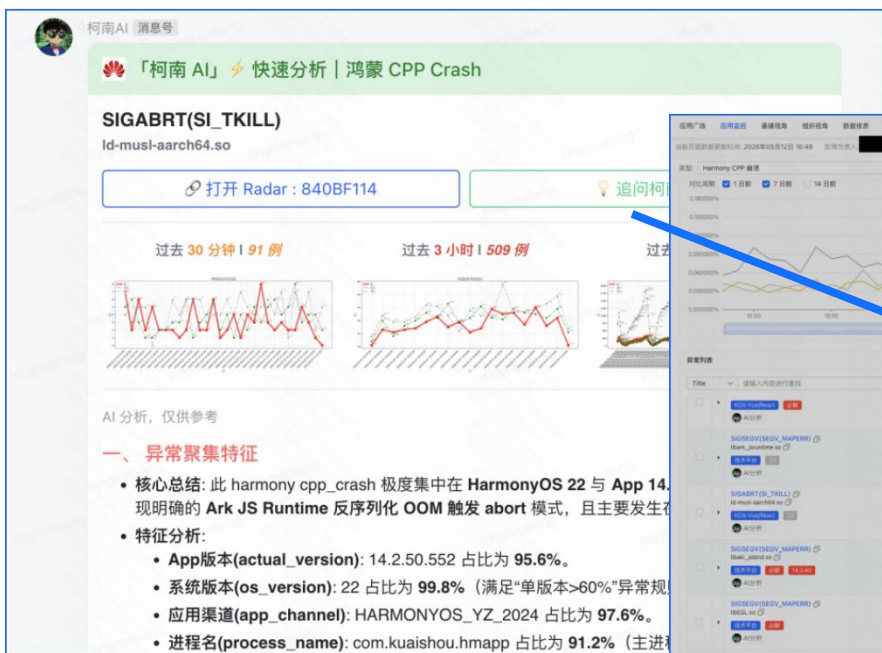
柯南 AI 定位



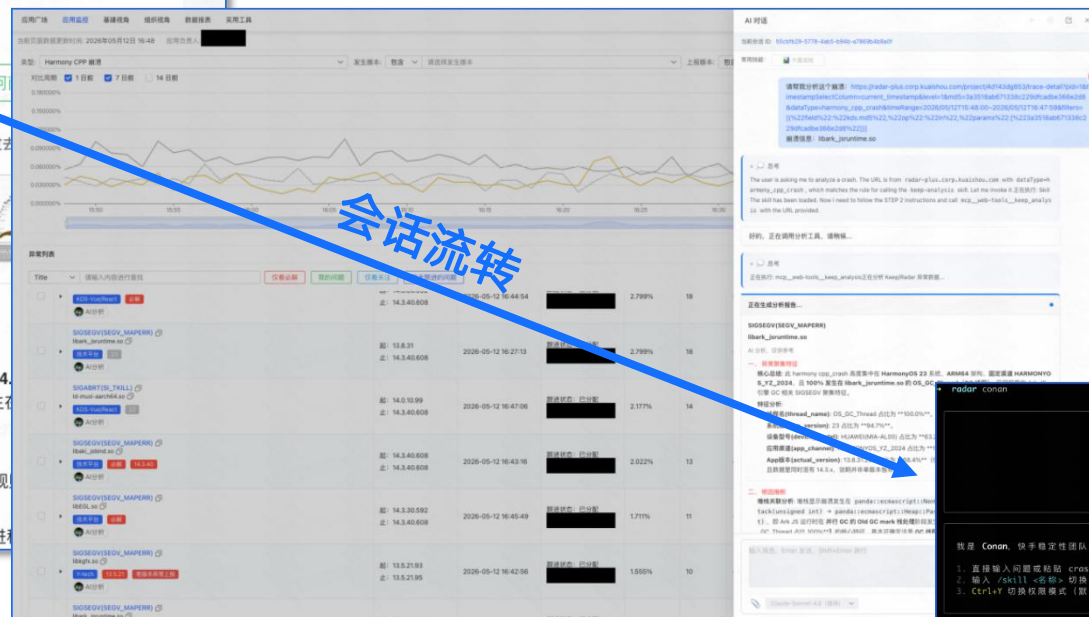
为什么选择问题处置环节



柯南 AI 产品形态 | 嵌入现有 workflow



IM 机器人



监控平台侧边栏

60% 渗透率 + 50% 采纳率

CLI



03

柯南 AI 的工程关键

五层架构 + 反馈迭代

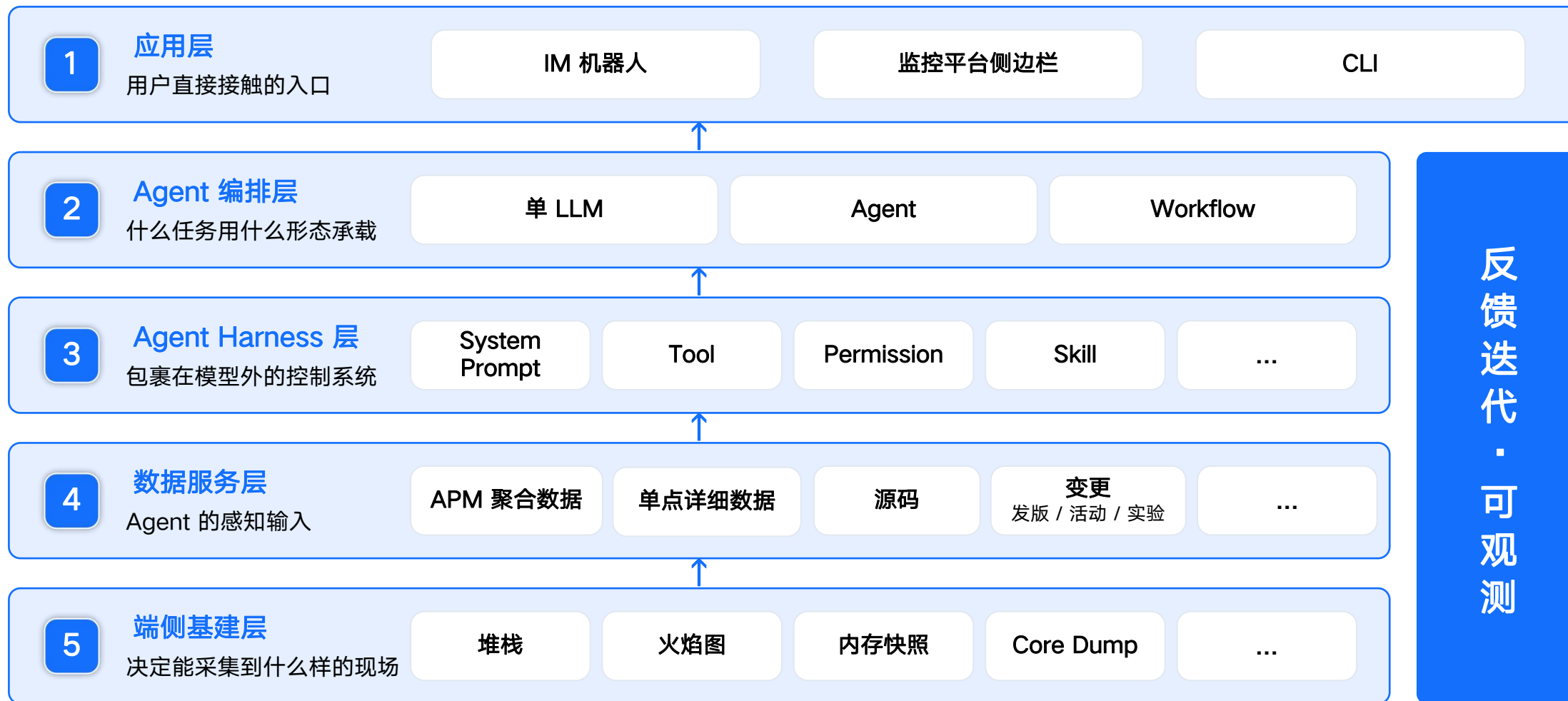
抛一个问题 | Harmony Cpp Crash 报警



只有系统堆栈 + 踩内存问题, 怎么查?

```
Reason:Signal:SIGSEGV(SEGV_MAPERR)@0x0027657461725f6b
Fault thread info:
Tid:8717, Name:..kuaishou.hmapp
#00 pc 00000000001ace0 /system/lib64/platformsdk/libuv.so(uv__async_io+416)
#01 pc 00000000002f6a4 /system/lib64/platformsdk/libuv.so(uv__io_poll+1060)
#02 pc 00000000001b7e4 /system/lib64/platformsdk/libuv.so(uv_run+888)(dc0e0
#03 pc 000000000073978 /system/lib64/platformsdk/libruntime.z.so(OHOS::Abil
#04 pc 000000000032cdc /system/lib64/chipset-sdk-sp/libeventhandler.z.so(7a
#05 pc 00000000001efe0 /system/lib64/chipset-sdk-sp/libeventhandler.z.so(OH
namespace)::EventRunnerImpl::ExecuteEventHandler(std::__h::unique_ptr<OHOS::
(7aa198768ac6fb3f322159dba72d6d)
#06 pc 00000000001b61c /system/lib64/chipset-sdk-sp/libeventhandler.z.so(OH
(7aa198768ac6fb3f322159dba72d6d)
#07 pc 00000000003fe64 /system/lib64/chipset-sdk-sp/libeventhandler.z.so(OH
#08 pc 0000000000cb6ec /system/lib64/platformsdk/libappkit_native.z.so(OHOS
#09 pc 000000000009fd4 /system/lib64/appspawn/appspawn/libappspawn_ace.z.so
#10 pc 00000000000ba3c /system/bin/appspawn(AppSpawnChild+512)(a0659f1195e6
#11 pc 000000000016ce4 /system/bin/appspawn(ProcessSpawnReqMsg+3788)(a0659f
#12 pc 000000000014b0c /system/bin/appspawn(OnReceiveRequest+632)(a0659f119
#13 pc 000000000017780 /system/lib64/chipset-sdk-sp/libbegetutil.z.so(Handl
#14 pc 000000000017098 /system/lib64/chipset-sdk-sp/libbegetutil.z.so(Handl
#15 pc 000000000014318 /system/lib64/chipset-sdk-sp/libbegetutil.z.so(Proce
#16 pc 000000000013ccc /system/lib64/chipset-sdk-sp/libbegetutil.z.so(RunLo
#17 pc 0000000000116e0 /system/bin/appspawn(AppSpawnRun+212)(a0659f1195e606
#18 pc 0000000000f2a4 /system/bin/appspawn(main+1148)(a0659f1195e606226bc6
#19 pc 0000000000b0800 /system/lib/ld-musl-aarch64.so.1(libc_start_main_sta
```

柯南 AI 架构



柯南 AI 架构



■ 确定性归代码，概率性归模型

代码

流程清晰 / 规则明确 / 答案唯一

模型

理解 / 推理 / 判断

效果稳定 + 成本可控

Workflow 案例 | 报警处置



■ 单 LLM 请求能解决的，不使用 Agent

多轮 Loop

5 – 10 轮 LLM 调用

Token 累积

上下文逐轮膨胀

Tool Call

概率失败 / 重试发散

Agent 失控

Loop 自决策 / 跑偏

单 LLM 案例 | 报警快速分析（维度分析）

INPUT

APM 维度聚合数据

设备 / 系统 / 地域 Top N · 占比 + 同比

OUTPUT

结构化维度归纳

集中维度 + 置信度 · 直接渲染卡片

LLM

单次请求 ×1

上下文一次聚合 · 60 秒 · 无多轮



Agent 案例 | 根因分析

①

长链推理

+ 跨工具调用

- ◆ 堆栈 / 源码 / 变更 / 聚合特征 / 历史 case
- ◆ 多次 Tool Call / 多个数据源 / 非固定流程

②

上下文动态扩展

+ Skill 按需加载

- ◆ 什么问题加载什么知识

```
▼ llm.completion #5
  🤖 deepseek-v4-pro
  ✓ Tool: Skill: keep-log-download
    https://keep.corp.kuaishou.com/exception-analysi
    s/crash/andr...
  ✓ Tool: Skill: keep-log-download
    https://keep.corp.kuaishou.com/exception-analysi
    s/crash/andr...

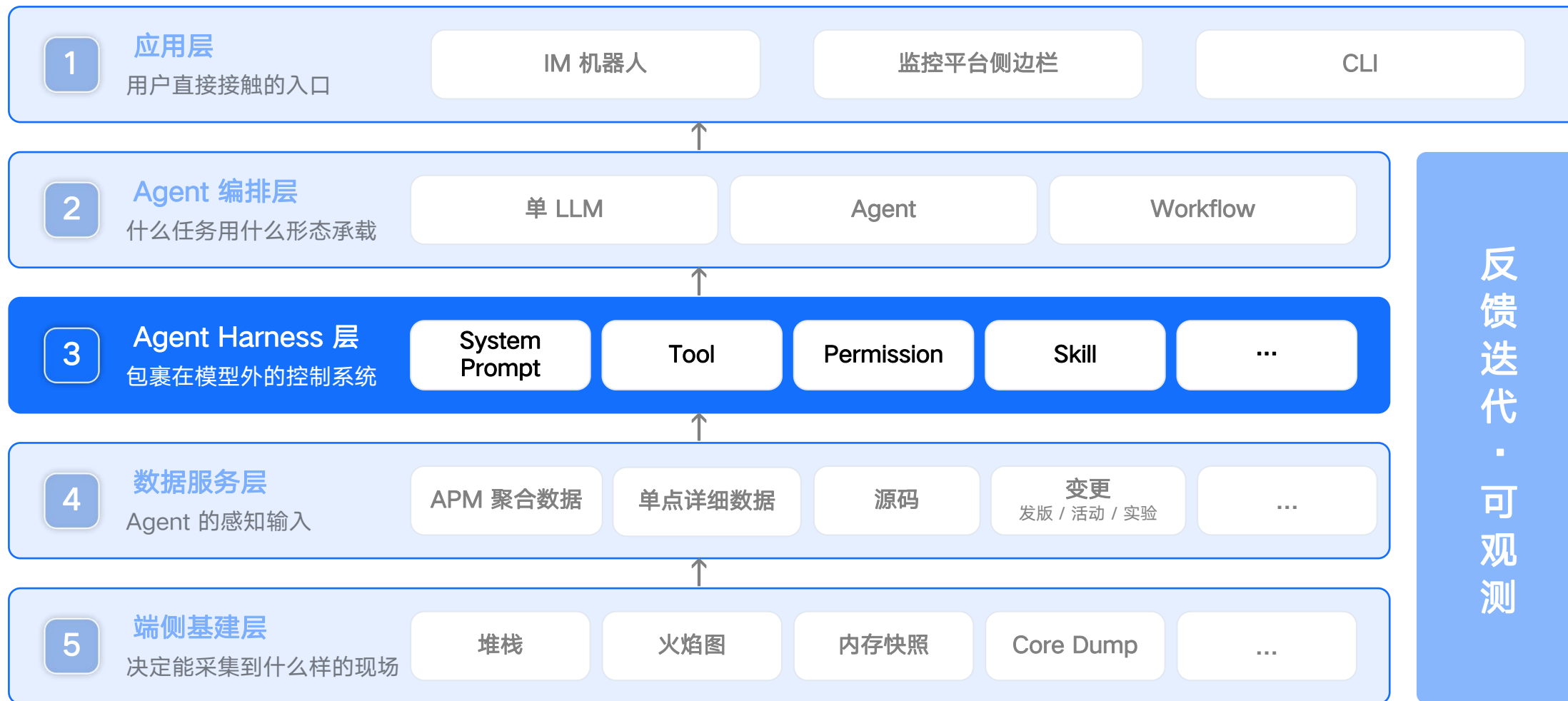
▼ llm.completion #6
  🤖 deepseek-v4-pro
  ✓ Tool: Bash
    uv run --with requests python3 /Users/zack/.cona
    n/skills/kee...
  ✓ Tool: Bash
    uv run --with requests python3 /Users/zack/.cona
    n/skills/kee...

▼ llm.completion #7
  🤖 deepseek-v4-pro
  ✓ Tool: Bash
    wc -l /tmp/crash_logs/main-8087-b1928b79-e4ea
    -46fb-ba27-6d57...
  ✓ Tool: Bash
    head -20 /tmp/crash_logs/main-8087-b1928b79-
    e4ea-46fb-ba27-6...

▼ llm.completion #8
  🤖 deepseek-v4-pro
  ✓ Tool: Write

▼ llm.completion #9
  🤖 deepseek-v4-pro
  ✓ Tool: Bash
    python3 /tmp/analyze_vma.py
```

柯南 AI 架构



■ Claude Agent SDK | 站在巨人肩膀上

理由① 工程红利

完整继承 Claude Code 生产打磨的 Harness

理由② 迭代同步

与 Claude 系列产品同步迭代

理由③ 适配深度

对 Anthropic 模型工程级适配

SDK 做 Agent runtime 通用工程

■ 轻量化定制 | system prompt + tool / permission

围绕领域诉求精简 system prompt

SDK 自带 prompt for 通用 Coding · 整段替换

① 剥离与领域无关的通用准则

Coding 约束 / auto memory / skill 清单等

② 注入领域硬规则

猜测 / 结论分段 · 置信度标注 · 禁跨类型对比

Tool 精简 + Permission 沙箱

失控半径压到最小

① Tool 精简

通用工具用 disallowedTools 关闭 · 缩小攻击面

② 物理沙箱锁定 cwd

Read / Write / Edit 锁 cwd · 越界即拒绝

Skill | 承接专家经验

为什么是 Skill?

判断① 按问题类型按需触发

渐进式披露 + 按需加载 · 原生匹配该形态
→ 30+ skill / 19K+ 行知识共存不撑爆上下文

判断② 对领域专家友好

MD + frontmatter · 不需懂 runtime / 不写代码
→ 「专家维护」从工程团队边界释放

Skill 如何组织?

原则① 不耦合 Agent · 独立迭代

skill-tree 单仓库 · 由领域专家维护
→ 独立发布 / 灰度 / 回滚

原则② 按异常退出类型划分

android-* / ios-* / harmony-* 各自独立
→ 宁愿适度重复 · 避免上下文污染

Skill 怎么写 | 私域知识注入

字段语义已与上报时不一致

典型 · Fd OOM 判定

APM 定界字段，Android 高版本上失效

∨ ## 禁止行为

- ****禁止****信任 `mCrashType`` / ``异常类型`` 等上报字段
- ****禁止****信任 `NATIVE_THREAD_OOM`` / `NATIVE_COMMO`

私有框架 / SDK 专有知识

LLM 训练语料从未见过，必须显式注入

典型 · `ekko fake_exception` 帧跳过

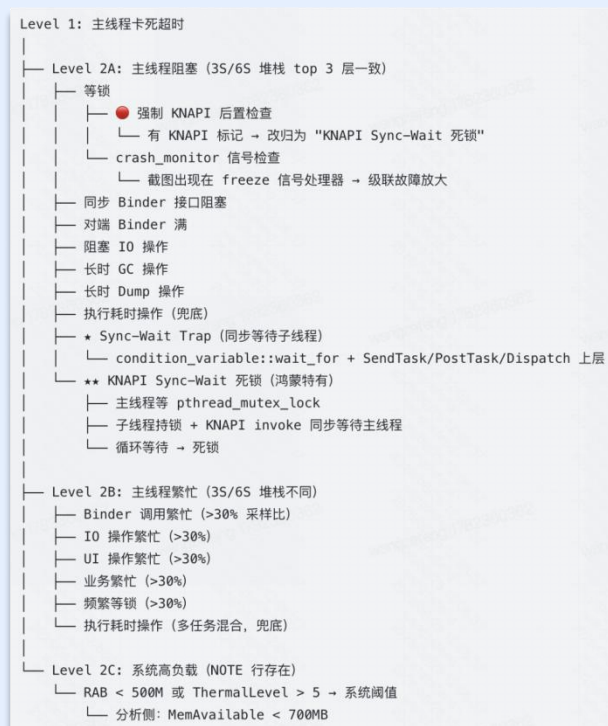
`ekko` 用于崩溃兜底，和崩溃本身业务逻辑无关

```
> **! fake_exception: 跳过 ekko 帧**: `fake_e  
兜底框架自身的帧 (如 `ekko_xxx`、`-[EkkoXxx xxx]  
示 ekko 执行了恢复逻辑，不提供业务信息。代码定位时*  
**不含 `ekko` 的业务帧**开始分析。ekko 本身不是问
```

Skill 怎么写 | 根因分析 SOP

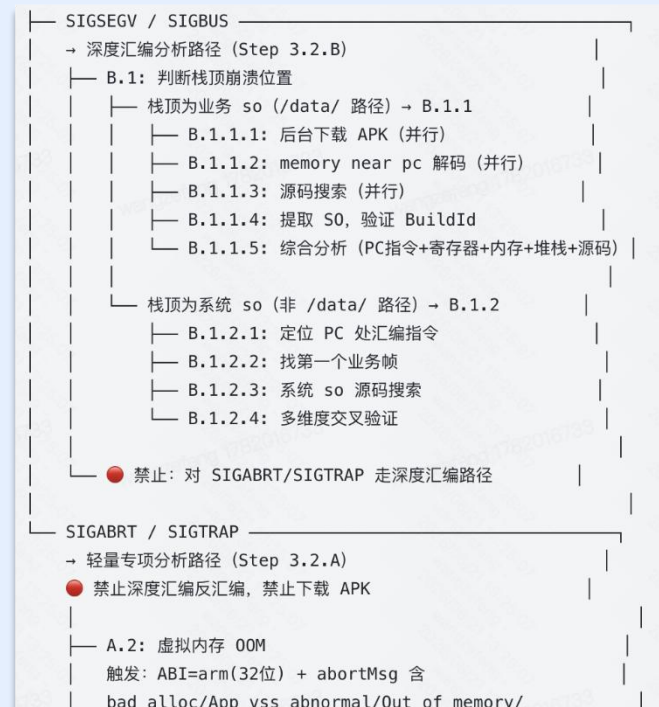
Case A · Harmony AppFreeze 主线程根因

3S/6S 双采样一致性 · 阻塞 vs 繁忙



Case B · Android Native Crash 信号路由

SIGABRT/SIGTRAP 走 abortMsg
SIGSEGV/SIGBUS 走深度分析



■ 瓶颈 & 探索项 | 模型 Loop + Memory

当前问题 — 根因分析能力卡在哪里

- ◆ 思考深度不足 (eg. NPE 只能回答哪里为空的问题)
- ◆ 信息缺失时本能幻觉 (缺证据时倾向"补全")



探索方向 ① 模型能力 + Agent Loop 工程

短中期 · Skill 强化约束 + 问题分级路由
长期 · Dynamic Workflow / 树形搜索

当前问题 — 修复效果落后于排障效果

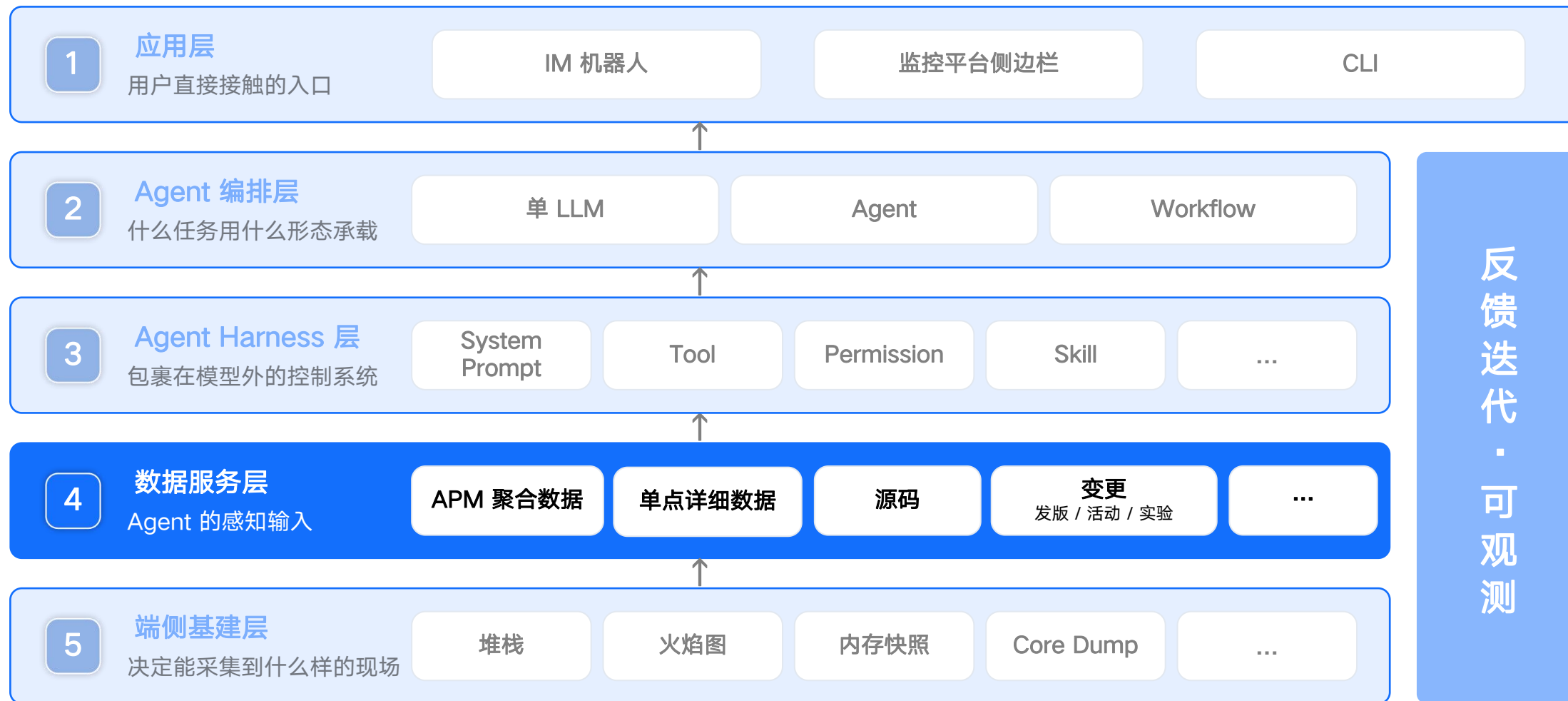
- ◆ 历史经验缺位: 同堆栈上次怎么修, Agent 不知道
- ◆ 业务暗知识只在人脑里 (能否动), 不在堆栈也不在文档里



探索方向 ② Memory 体系

Memory 承载 · 历史经验 + 业务暗知识
业务暗知识持续显性化

柯南 AI 架构



■ 数据服务 | 以源码服务为例

为什么需要独立的「源码服务」？而不是让 Agent 直接 git clone

理由① 解决「仓库找不到」

一个崩溃堆栈帧可能横跨 5-10 个仓库

- ◆ 模块名 <-> 仓库名 — 往往不一致
- ◆ 包名对应哪仓库 / 哪分支 / 哪 commit — 光看堆栈不知道

→ 源码服务做仓库聚合 + 索引

理由② 解决「拉取/检索」效率

git clone 大仓库每次几十秒到几分钟

- ◆ Agent 每查一次代码都重新拉取 — 时延不可接受
- ◆ 源码服务把所有仓库预聚合好

→ Agent 搜索像本地 grep 一样快

■ 数据消费 | Agent 友好接口

把「人 + GUI」的数据 → 重新包装为「Agent + bash 接口」

完整 bash 接口

```
$ git grep "Class::method"  
$ git log --oneline -- src/file.cpp  
$ git blame src/file.cpp
```

- ◆ LLM 天然熟悉 bash 语义
- ◆ 无需单独定义 API spec

安全性保证 · 灵活性给足 / 边界锁死

✓ 容器只读挂载

✓ 命令超时

✓ 完整命令日志

→ Agent 自由探索 · 但不能越界

■ 源码服务的下一步

方向① 从关键字 → 语义

当前 — 关键字全文搜索 (grep)

- ◆ 空指针 — 跨函数追踪对象生命周期
- ◆ 跨语言 — KMP / NAPI 桥接关系

探索 LSP / CodeGraph

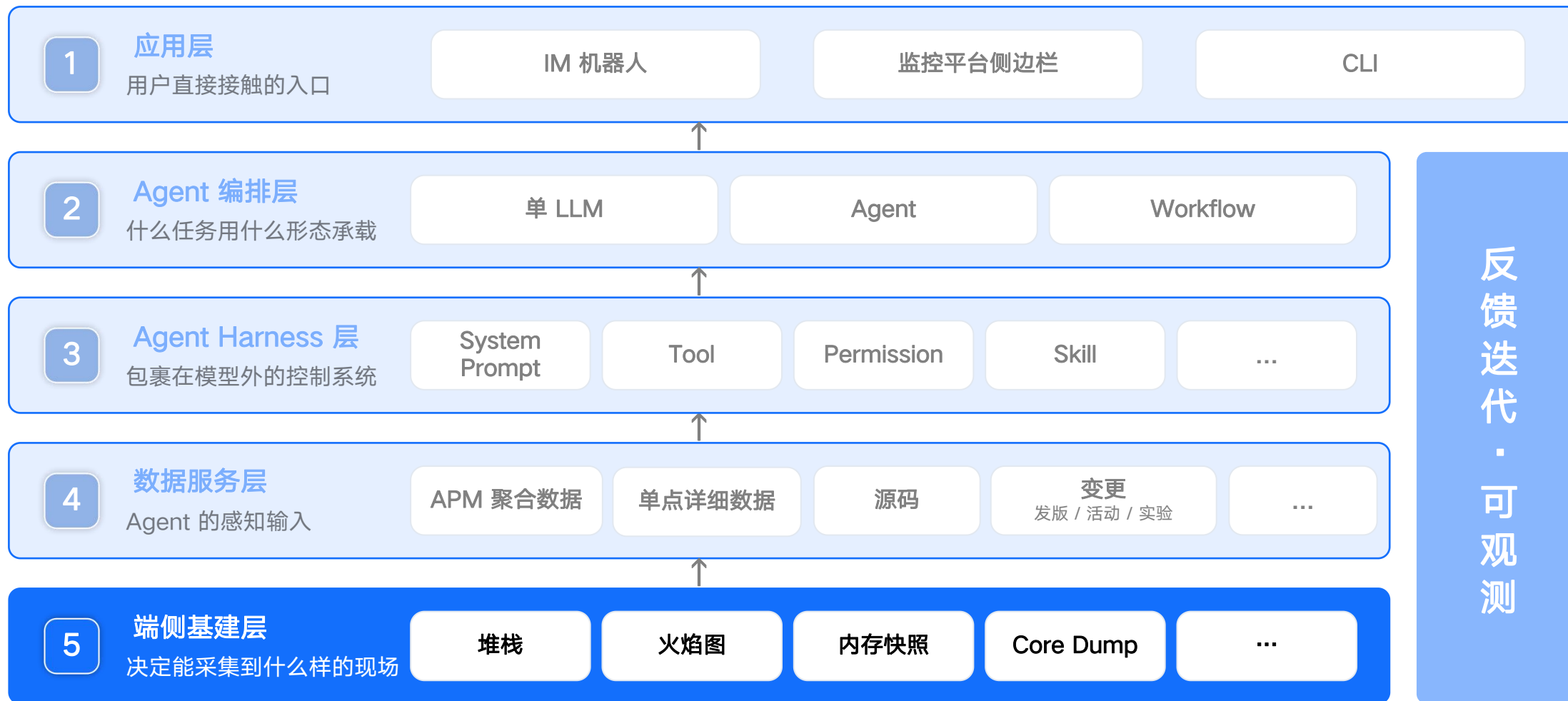
方向② 单体 → 分布式

当前 — 单体架构 日调用 X 万次

- ◆ 已遇扩展性瓶颈
- ◆ 覆盖率 + 接入场景持续增长

探索 分布式架构

柯南 AI 架构



■ 端侧基建 | 上下文决定效果上限



Agent 无法凭空创造信息

端侧基建= Agent 体系的「地基」

决定 Agent 能看到什么



决定 Agent 能做到什么

■ 三端 Core Dump 探索 | AI 时代的 ROI 重写

典型场景 — ART JIT 踩内存 (PC 落在 jit-code-cache / tombstone 不够)

过去 — 难以被消费

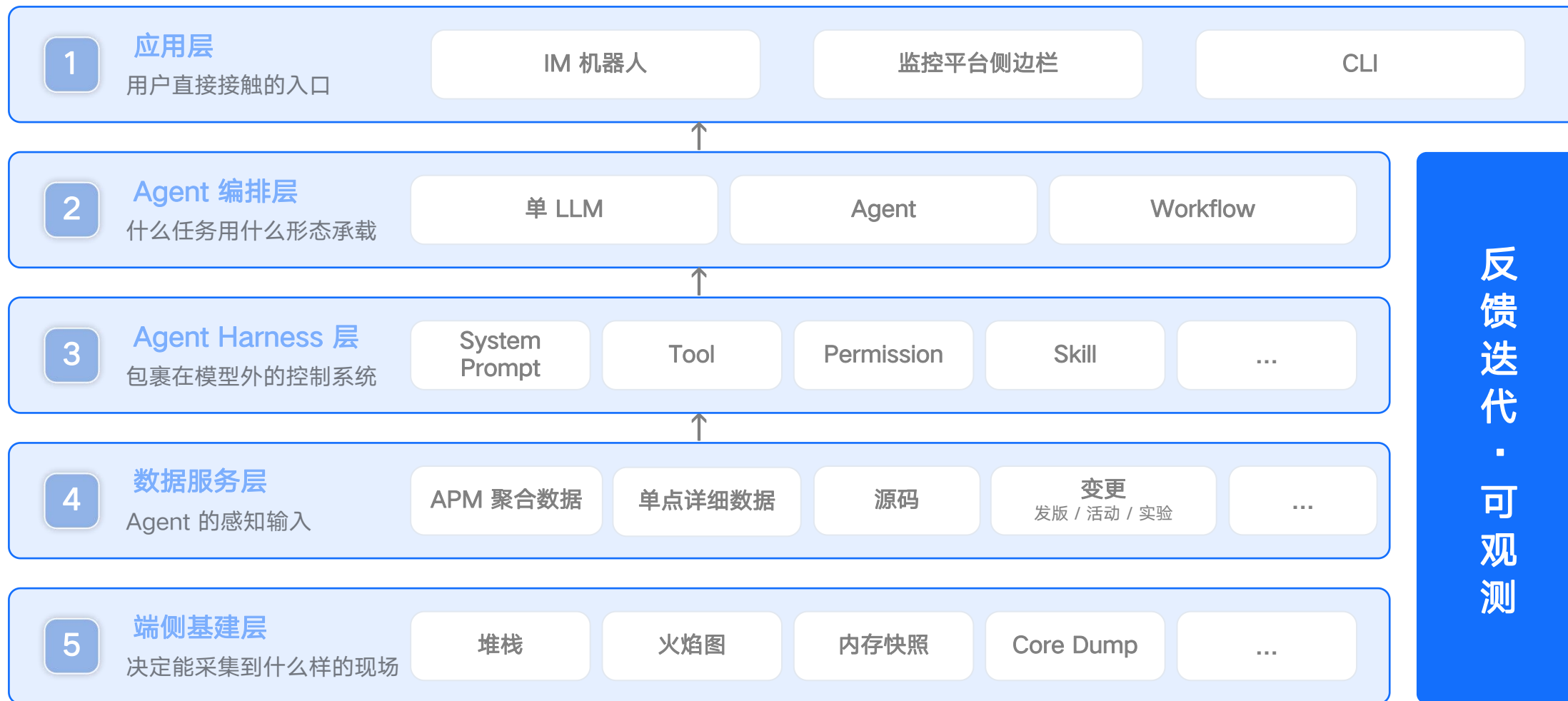
- ◆ Dump 文件几百 MB 起步
- ◆ 必须 专家 + lldb / gdb

AI 时代 — Agent 接管挖掘

- ◆ 自动反汇编 + 关联其他信息
- ◆ 任何研发 + 一个 Agent

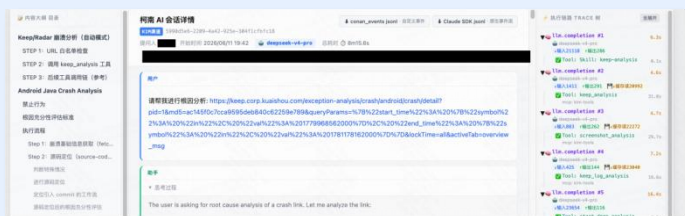
端侧基建从「为人优化」→「为 Agent 优化」

柯南 AI 架构



全链路观测体系 | 可观测、可评测、可运营

从单次调用 → 到整体运营 — 全链路观测一体化



① 链路层 | Agent 一次调用过程完整还原

对话 → 大模型 → 工具调用 / Skill 按时序还原



② 指标层 | 成本和性能记清算准

4 类 Token 消耗 · 模型降级状态 · 首 Token 响应速度



③ 运营层 | 业务视角盯产品健康度

用户增长 · Token 消耗 · 模型调用 · 会话管理

Badcase 驱动机制

解决一对核心矛盾

AI 「不知道自己不知道」 × 人「不知道自己知道」

幻觉

缺乏引导

① case 管理

采纳率标准

case 标注平台串起三步流程：

- ◆ 分发 — 筛有效问题
- ◆ 回访 — 业务反馈 + 专家校验
- ◆ 标注 — 沉淀评测 case · 识别优化项

② case 总结

- ◆ 采纳率通晒
- ◆ 细粒度 case 总结改进
- ◆ 共识优化项 + 落地

前面的问题 | Harmony Cpp Crash 报警



只有系统堆栈 + 踩内存问题, 怎么查?

```
Reason:Signal:SIGSEGV(SEGV_MAPERR)@0x0027657461725f6b
Fault thread info:
Tid:8717, Name:..kuaishou.hmapp
#00 pc 000000000001ace0 /system/lib64/platformsdk/libuv.so(uv__async_io+416)
#01 pc 000000000002f6a4 /system/lib64/platformsdk/libuv.so(uv__io_poll+1060)
#02 pc 000000000001b7e4 /system/lib64/platformsdk/libuv.so(uv_run+888)(dc0e0
#03 pc 0000000000073978 /system/lib64/platformsdk/libruntime.z.so(OHOS::Abil
#04 pc 0000000000032cdc /system/lib64/chipset-sdk-sp/libeventhandler.z.so(7a
#05 pc 000000000001efe0 /system/lib64/chipset-sdk-sp/libeventhandler.z.so(OH
namespace)::EventRunnerImpl::ExecuteEventHandler(std::__h::unique_ptr<OHOS::
(7aa198768ac6fb3f322159dba72d6d)
#06 pc 000000000001b61c /system/lib64/chipset-sdk-sp/libeventhandler.z.so(OH
(7aa198768ac6fb3f322159dba72d6d)
#07 pc 000000000003fe64 /system/lib64/chipset-sdk-sp/libeventhandler.z.so(OH
#08 pc 00000000000cb6ec /system/lib64/platformsdk/libappkit_native.z.so(OHOS
#09 pc 000000000009fd4 /system/lib64/appspawn/appspawn/libappspawn_ace.z.so
#10 pc 00000000000ba3c /system/bin/appspawn(AppSpawnChild+512)(a0659f1195e6
#11 pc 0000000000016ce4 /system/bin/appspawn(ProcessSpawnReqMsg+3788)(a0659f
#12 pc 0000000000014b0c /system/bin/appspawn(OnReceiveRequest+632)(a0659f119
#13 pc 0000000000017780 /system/lib64/chipset-sdk-sp/libbegetutil.z.so(Handl
#14 pc 0000000000017098 /system/lib64/chipset-sdk-sp/libbegetutil.z.so(Handl
#15 pc 0000000000014318 /system/lib64/chipset-sdk-sp/libbegetutil.z.so(Proce
#16 pc 0000000000013ccc /system/lib64/chipset-sdk-sp/libbegetutil.z.so(RunLo
#17 pc 00000000000116e0 /system/bin/appspawn(AppSpawnRun+212)(a0659f1195e606
#18 pc 00000000000f2a4 /system/bin/appspawn(main+1148)(a0659f1195e606226bc6
#19 pc 00000000000b0800 /system/lib/ld-musl-aarch64.so.1(libc_start_main_sta
```

前面的问题 | Harmony Cpp Crash 报警



根本原因

崩溃类型: Use-After-Free (UAF) — libuv async handle 被关闭/释放后仍有 pending 事件在事件队列中, `uv_async_io` 访问已释放内存导致 SIGSEGV。

触发链路:

1. RN Bundle 加载失败: [REDACTED]
时, RCTView 组件未在 UIManager 中注册, 触发 JS 侧 `Invariant Violation`, 导致 RN 视图创建失败。
2. `CatalystInstance` 销毁: 加载失败触发 `CatalystInstanceImpl.destroy()`, 在主线程上同步

• 置信度: 高 (寄存器 ASII 覆写 + 业务日志时序 + 源码分析)

- **Signal:** SIGSEGV(SEGV_MAPERR), `faultAddr=0x006e756f43657481` (含 ASCII 字符 `nuoCet`, UAF 典型特征)
- **寄存器:** `x0=0` (NULL 指针), `x9=0x746e756f43657479` ("tCounty" ASCII — 同源 JSON 字符串覆写)

• 关键时序 (应用日志):

- `06:55:43.009` — RN 加载 [REDACTED] bu
- `06:55:43.031~043` — JS 侧多次 `Invariant Violation`
"RCTView" was not found in the UIManager

• HiLog 噪声排除: 大量 "Promise not init" + UnHandle 关噪声, 不出现在崩溃线程上, 不作为根因线索

- `06:55:43.133` — `CatalystInstanceImpl.destroy()`
`Thread::quitSynchronous` (js/native_modules 线程退
- `06:55:43.158` — 崩溃发生

工程关键小结

①

确定性归代码，概率性归模型

Workflow / 单 LLM / Agent 各适其用

效果稳定 + 成本可控

②

基于 Claude Agent SDK

Skill 承接专家经验

轻量化 + 领域化

③

Agent 友好的数据提供形式

Bash 接口

LLM 天然熟悉 bash

④

上下文决定效果上限

端侧基建 = 地基

Agent 看不到 = 永远做不到

⑤

反馈迭代决定走多远

Badcase 驱动

决定 Agent 的长期表现

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



04

总结与展望

Agent First, Human in Control

■ 总结 | 角色升级

稳定性专家：从 「亲自处理每个问题」 → 升级为 「建设能处理问题的系统」

① 专家经验，第一次有机会规模化

排障 = 推理密集型任务 · AI 让专家经验放大输出

- ◆ 亲自上手 → 沉淀 Skill / 共建 Context
- ◆ 角色：前线救火员 → 消防系统设计师

② Context 是壁垒

是享受模型升级「免费午餐」的前置条件

- ◆ 模型会持续变强
- ◆ 没有 Skill / 端侧基建 打底，模型再强也跑不动
- ◆ 长期建设 · 形成复利



展望

Agent 工程深化 × 上下文工程复利 × 反哺稳定性治理 —— 让 AI 把人的影响半径再放大

① Agent 工程深化

突破长尾采纳率瓶颈

- ◆ loop verify
- ◆ Memory

② 上下文持续复利

享受模型升级红利

- ◆ 端侧基建持续深化
- ◆ 语义检索
- ◆ 私域 SOP 持续沉淀

③ 反哺稳定性治理

从问题处置 → 治理优化

- ◆ 专家：做体系建设
- ◆ 一线：做更深优化

柯南 AI 不是终点 —— 是「用 AI 打败 AI」新阶段的起点

THANKS

探索 AI 应用边界

Explore the limits of AI applications